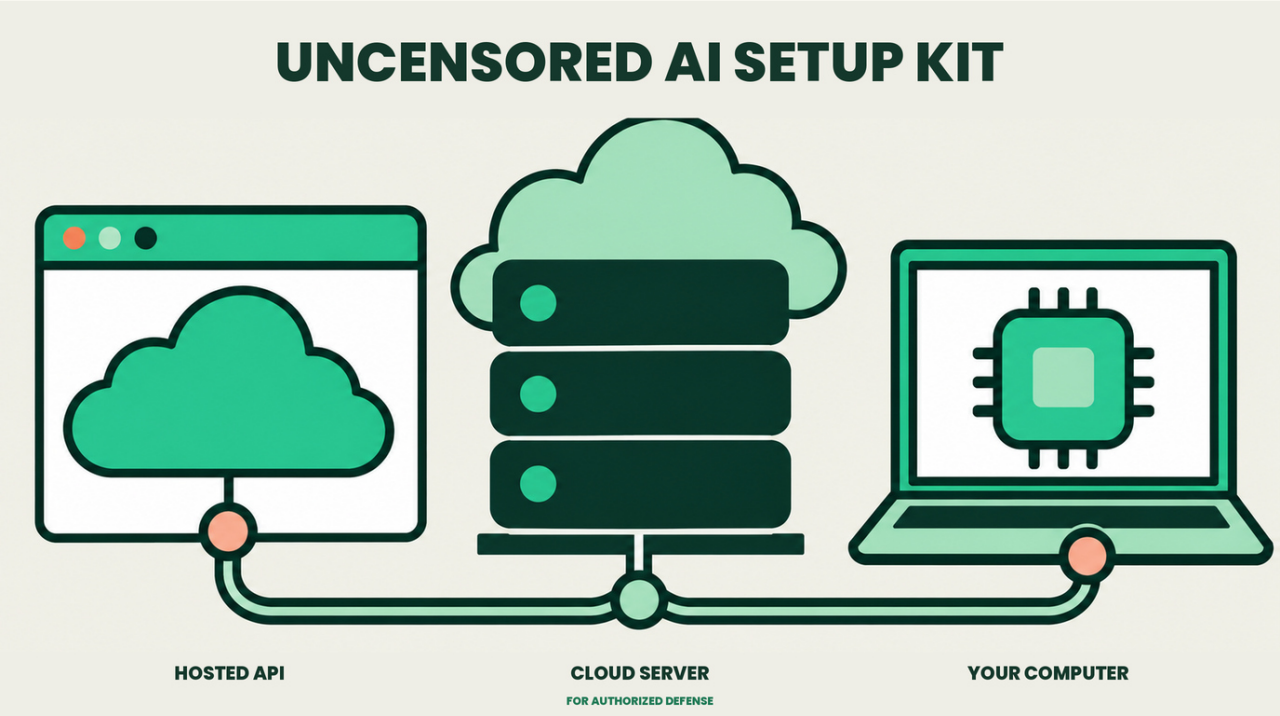
Uncensored AI Setup Kit

Three ways to run your own AI. One place to start.



Prompts, guides, a local practice lab and verification tools.

Use only on systems you own or have explicit permission to test. Keep credentials private and verify AI findings before acting.

Open the public repository and copy-ready files

Start here

Defensive use only. Work on systems you own or have explicit permission to test. Keep credentials private and verify AI findings before acting.

This kit helps you connect a model to DeepSeek Harness using a hosted provider, a rented RunPod GPU, or LM Studio on your own computer. It also includes a small local lab and checks that tell you whether the connection actually works.

Fastest route

1. Read Choose your route. Hosted is usually the fewest setup steps; local avoids a separate inference bill if your hardware can run the model.

2. Open ASSISTANT-SETUP.md, copy its prompt, and give your assistant this repository or unpacked folder.

3. Use the numbered prompt files in order for your chosen route. Enter secrets directly in the app or a private file, never in chat.

4. Run the model checks and the local lab before connecting tools to anything valuable.

5. For RunPod, test the cost guard and confirm the actual stopped state. Storage can still cost money after compute stops.

What you need

An assistant capable of reading files and helping with setup, a supported Node.js version for Harness/MCP, and Python 3.10+ for the included helpers. Follow each upstream project's current OS/runtime requirements. A paid AI subscription does not automatically include hosted API credits or a GPU rental.

Files you'll use

Path	Purpose
`prompts/`	Nine copyable resource-edition prompts with safety boundaries
`recorded-prompts/`	Recovered original cloud/local messages and spoken excerpts; provenance is explicit
`guides/`	Three routes, tools, model safety, troubleshooting and cost control
`config/`	Credential-free provider fields and Harness MCP configuration
`scripts/check_endpoint.py`	Real two-turn response and harmless function-call checks
`scripts/local_lab.py` / `safe_check.py`	Synthetic localhost-only header-check exercise
`scripts/idle_proxy.py`	Optional local cost guard for an existing RunPod endpoint
`verification/STATUS.md`	What was tested and what still depends on your account
`downloads/quick-start.pdf`	Readable companion guide; Markdown remains editable

No model weights, paid services, production data, or third-party credentials are included. The scripts never provision a paid GPU. The model recommendations are starting points dated 26 October 2026, not guarantees of safety or speed.

Choose how to run your model

Defensive use only. Work on systems you own or have explicit permission to test. Keep credentials private and verify AI findings before acting.

Route	Good fit	What you pay for	Your main responsibility
Hosted API	Fast setup; no capable local GPU	Provider API usage; subscription and API billing can differ	Provider trust, key handling, usage limits
RunPod	More control over model/runtime	GPU time plus storage and any additional quoted charges	Patching, authentication, uptime and shutdown
Local / LM Studio	Compatible machine with enough spare memory	Hardware, power and time; no rented GPU for local inference	Downloads, runtime updates and local permissions

Choose hosted when avoiding infrastructure is worth the markup. Choose cloud when you need a particular model beyond your machine's capacity. Choose local when your machine fits it and you want direct control over the runtime. None makes a model automatically safe or more capable.

Start with text. Add one harmless function call. Add screenshots only when needed, and verify each layer separately. These checks catch a common trap: a model can answer a greeting while failing the exact tool or image feature you need.

If you already have one working route, keep it as a fallback while trying another. Do not replace working settings just to match this video.

Route 1: hosted API

Defensive use only. Work on systems you own or have explicit permission to test. Keep credentials private and verify AI findings before acting.

Before starting

Use prompt 01 to prepare the Harness. Then use prompt 02. You need a provider account that permits your intended defensive workload and supports a model/protocol compatible with Harness. The provider can have identity checks and separate API billing.

Connect

1. Open the provider's official docs and copy its API base URL and wire protocol. The video shows Abliteration and OrcaRouter; current availability and terms must be checked at their own sites.

2. Create a limited key, set a usage limit where offered, and enter the key directly in Harness's provider form or its private credential store.

3. In Harness open Settings > Models > Add model provider > Custom model API. Use `config/provider-fields.json` as a worksheet, not a file to import.

4. Fetch available models or enter the exact ID from the provider. Display names and IDs can differ. Select a model with the capabilities you intend to test.

5. Start a new session in the test workspace. Old sessions may retain their original model.

Verify

Run `python3 scripts/check_endpoint.py --base-url https://YOUR_PROVIDER/v1 --model YOUR_MODEL_ID` with `MODEL_API_KEY_FILE` pointing to your private key file. This sends only synthetic prompts. It consumes a small amount of API usage. Use `--tools` to test function-call syntax and `--image assets/vision-test.png` for image input. Inspect the result and repeat the harmless tool workflow through Harness itself.

Stop / undo

Close the test session. Disable or remove only the provider entry you added, and revoke the test key if no longer needed. Check the provider usage dashboard. Uninstalling Harness does not necessarily cancel API billing or revoke a key.

Sources: Harness model settings, Abliteration, OrcaRouter.

Route 2: a rented cloud GPU

Defensive use only. Work on systems you own or have explicit permission to test. Keep credentials private and verify AI findings before acting.

Before renting

Run prompt 04. Ask the assistant to distinguish model weights, vision projector, inference runtime, context size and memory overhead. The biggest affordable weights are not always the best working tool model. See the dated recorded setup before choosing.

Approve a written quote with the exact GPU, quantity, region, hourly rate, persistent storage size/rate, maximum runtime and total budget. A quote is not a guarantee that a GPU is available later. Do not grant open-ended authority to keep retrying larger GPUs.

Deploy the approved plan

Run prompt 05 after connecting the RunPod account through its supported plugin or CLI. Use a trusted inference template or a pinned runtime. Put model files, the compatible projector and startup configuration under persistent storage. Keep temporary caches separate. The included kit does not rent GPUs automatically.

Expose only an authenticated model endpoint, not an unauthenticated shell or admin console. Keep the RunPod management key distinct from the inference key. Confirm the endpoint refuses a request without the inference key.

In Harness create a custom API provider. Use the returned `/v1` endpoint, protocol and exact served model ID. If using the local cost guard, Harness points at the local guard instead of the remote model endpoint.

Acceptance tests

- Text response and two conversation turns.

- Harmless function call plus a follow-up after the tool result.

- Synthetic screenshot if image input is part of the model choice.

- An unauthenticated request fails.

- Stop, then confirm EXITED in RunPod; restart, confirm the model/config still exist, and repeat the harmless request.

Use the included endpoint checker, then verify through Harness. Record failures. The public release has no claim of a fresh paid deployment in every reader's account.

End the session

Follow the cost-control guide. Stop this exact Pod when finished. Review storage charges separately. Delete it only after deciding what to keep and making a backup. A container-disk file may disappear on stop; a Pod volume disappears on termination. Verify the current behavior in RunPod storage docs.

Sources: RunPod Pods, Pricing, Storage.

Route 3: LM Studio on your computer

Defensive use only. Work on systems you own or have explicit permission to test. Keep credentials private and verify AI findings before acting.

Choose and load

Use prompt 07. Keep headroom for your OS, other apps, model context/cache and vision projector. Check available memory, not only total RAM. Read the model card and license; do not treat download counts as a safety certificate.

Use prompt 08 to connect the model you chose. The recording used a Huihui Qwen3.6 35B-A3B GGUF build; it is not suitable for every computer. See `config/recorded-models.json` for recovered source identity. Reuse a compatible download instead of downloading again.

Open LM Studio, load the chosen model, and start its local server from the Developer tab. Keep network serving off. Its CLI also supports:

```
lms ls
lms load YOUR_MODEL_KEY --context-length 4096 --identifier defensive-local --ttl 600
lms server start --bind 127.0.0.1 --port 1234
```

The context value is a conservative starting example, not the recorded setting. If load fails, use a smaller model/context or close other applications. Do not force allocation beyond available memory.

Image compatibility note

The fresh test of the existing Q4_K runtime passed text and function calling but failed the synthetic image check. Do not use that result as proof of working vision. If you need screenshots, confirm the correct projector and runtime support, then pass the image test before continuing. See verification/local-model-result.json.

Connect Harness

Settings > Models > Add model provider > Custom model API. Use OpenAI Chat Completions and `http://127.0.0.1:1234/v1`, then select the ID returned by the server. Bind the Harness UI locally too.

```
python3 scripts/check_endpoint.py --base-url http://127.0.0.1:1234/v1 --model
defensive-local --tools --image assets/vision-test.png
```

This tests a two-turn conversation, a harmless function call and the synthetic image separately. It does not grant the model shell access. After that, test a harmless Harness tool on the local lab. Failing one capability is a reason to fix it or choose another model, not to mark the setup complete.

Close down

```
lms unload defensive-local
lms server stop
```

Stop only the server you started and unload only your test model. If you were already running other workloads, preserve them.

Sources: LM Studio server, Tool use, Harness provider guide.

Add tools and prove they work

Defensive use only. Work on systems you own or have explicit permission to test. Keep credentials private and verify AI findings before acting.

Use prompt 03 after the basic model works. A model is the decision-maker; tools provide browser, research and scanner capabilities. Keep each tool's privileges and inputs narrow.

Harness configuration

`config/dsh-mcp.patch.yml` follows Harness's native MCP plugin shape. It is a reference overlay, not a replacement for your full profile. Have your assistant merge it with the active profile after checking your installed version. Do not overwrite provider settings. The generic JSON example is for other clients that explicitly accept `mcpServers`.

Three small checks

1. Start `python3 scripts/local_lab.py` in a separate terminal. In Harness ask Playwright to open `http://127.0.0.1:8765`, report the page title and read the synthetic welcome text. Use an isolated browser session. Do not connect your personal signed-in browser for this test.

2. Ask Firecrawl to read the public Harness README and return its title and official documentation link. Its current limited keyless endpoint can cover the first test; full features need OAuth or a private key. Third-party research traffic is subject to provider terms and quotas.

3. Run `nuclei -version`. Review `examples/nuclei-local-headers.yaml`, then run that one passive GET template against the local lab. Do not load an unreviewed template collection against real targets.

```
nuclei -u http://127.0.0.1:8765 -t examples/nuclei-local-headers.yaml -rl 1 -c 1
```

The template checks a response header; it does not exploit an application. Our Python safe checker is an even smaller way to practice the same evidence/report process. The model must not decide its own scope by reading a webpage.

Sources: Playwright MCP, Firecrawl MCP, Nuclei, Harness MCP.

RunPod cost control

Defensive use only. Use your own approved Pod. Keep credentials private.

Stopping an app is not the same as stopping a GPU. Always confirm the Pod is stopped in RunPod. Persistent storage can still be billed.

Start with a manual limit

Write down the exact Pod ID/name, approved hourly rate, storage charges, and stop time. Configure any available account budget alerts. At the end, stop that Pod in the console and read back its status. Do not delete it until you've decided what to keep.

Optional local idle guard

`scripts/idle_proxy.py` forwards OpenAI-compatible requests and watches inactivity. It binds only to loopback, requires a local bearer token, does not log prompts or credentials, and starts in dry-run mode. It can stop an existing Pod only when explicitly enabled. It never starts, creates or deletes a Pod.

This is a helper, not a billing guarantee. It cannot see clients that bypass it. If the computer sleeps, the process crashes, or the network fails, it cannot stop the remote GPU. Use it alongside manual stop and provider-side budget controls.

1. Put the inference key in a private file and set `MODEL_API_KEY_FILE` to that path. Create a separate random local-proxy token in another private file. On macOS/Linux, use permission `600`. Never put keys in command-line arguments or the repository.

2. Set `LOCAL_PROXY_TOKEN_FILE` to the local token path. The Harness provider uses that local token, not the RunPod control key.

3. Start dry-run mode. Substitute the authenticated model endpoint returned by your deployment:

```
python3 scripts/idle_proxy.py --upstream https://YOUR_ENDPOINT/v1 --idle-seconds 900
```

4. Point Harness at `http://127.0.0.1:8766/v1`. Enter the local-proxy token privately in Harness. Route all inference traffic through this URL. Use the model's original ID.

5. Send one request, wait, then send another. The timer must restart. A running or streaming request must prevent stop. After the idle interval, the terminal reports `dry_run_would_stop`. Verify before enabling real control.

6. To opt in, set `RUNPOD_API_KEY_FILE` to a private management-key file and restart with the exact Pod ID and name:

```
python3 scripts/idle_proxy.py --upstream https://YOUR_ENDPOINT/v1 --idle-seconds 900
--enable-stop --pod-id YOUR_POD_ID --pod-name YOUR_EXACT_POD_NAME
```

The helper checks identity before requesting stop, then waits for `desiredStatus: EXITED`. It reports `stop_failed_or_unverified` on failure. Check the RunPod console yourself after the first real test. Do not treat a stop request alone as proof.

Restart and remove

Restart the Pod explicitly in RunPod, confirm its endpoint and authentication still work, then restart the local proxy. The helper deliberately does not auto-start a paid resource. To remove the helper, stop its process, restore the Harness provider's original endpoint, and revoke its dedicated control key. Retain your inference key only if you still need it.

The timer counts all forwarded model API requests, including model-list calls. Periodic discovery traffic can therefore keep a Pod awake. A stream is considered active until the upstream connection closes or times out. This guard is intended for a single user's dedicated Pod; it is not suitable for shared workloads without coordinating all clients.

Sources: Stop API, Pod status, Storage.

Before a downloaded model gets tools

Defensive use only. Work on systems you own or have explicit permission to test. Keep credentials private and verify AI findings before acting.

Read ProjectDiscovery: How abliterated models can get you pwned, the article promised in the video. It describes a deliberately backdoored research model. It does not establish that the models in this kit are compromised.

Treat weights and tools as separate trust decisions. Record the model publisher, repository, exact revision, file hash, format and license. Prefer a documented provenance chain; a familiar name or popular download can still be modified. A file scanner cannot certify model behavior.

First run a model without tools and without personal files. Then grant the smallest tool set needed for one synthetic task. Prefer a disposable, isolated environment. Enforce restrictions in the OS/container/runtime: telling a model not to read secrets is not a filesystem boundary. A localhost bind also does not isolate the model from files the harness can access.

Keep real credentials and production data out of the test workspace. Give research tools public documentation URLs; do not upload private code or customer records to a third-party crawler. Validate tool arguments before execution and review unexpected network destinations.

Watch for requests to fetch and execute remote scripts, expose keys, disable protections, or expand scope. Stop and investigate. Do not test malicious payloads from the article on your working machine.

Keep the model card and runtime version with each test result. If you update the weights, projector, template, runtime or tool permissions, rerun the relevant checks.

Also read Harness safety guidance and Hugging Face model security.

Fix the smallest broken layer

Defensive use only. Work on systems you own or have explicit permission to test. Keep credentials private and verify AI findings before acting.

Symptom	Check first	Next action
Connection refused	Is the server running at the right port?	Run server status; keep localhost settings consistent
401 or 403	Correct key and account permissions?	Enter the key privately; do not print it in a report
Models list empty	Does the endpoint implement `/v1/models`?	Check provider docs and manually confirm the exact model ID
Greeting works, tools fail	Does this model/runtime support tool calls?	Run the harmless function test; inspect the chat template
Screenshot ignored	Is vision enabled with a matching projector?	Run the synthetic-image check separately
Model fails to load	Enough free memory and compatible runtime?	Reduce context or model size; do not repeatedly force OOM
Harness keeps old model	Existing sessions can keep their model	Start a new test session and confirm selection
MCP listed but no result	Did a real tool call succeed?	Inspect the server startup and one bounded test
Cloud still billed	Compute stopped, or just the app?	Confirm the Pod is EXITED; review storage charges separately
Idle guard did nothing	Did all requests pass through it; was it awake?	Keep manual stop/budget controls and inspect the guard status

If a change fails, roll back only what you changed. Preserve working profiles and original files. Never solve a connection issue by exposing an unauthenticated endpoint or pasting a key into a public issue.

Verification status

Release preparation: 26 October 2026. These are bounded tests, not a security certification.

Layer	Status
Offline helper suite	11 tests passed: denied scope, remote target rejection, expired scope, synthetic finding/fix, proxy auth/origin restrictions, idle state and failed stop handling
Local LM Studio	Two-turn memory and harmless function roundtrip passed; synthetic image test FAILED on the loaded Q4_K runtime. See local-model-result.json.
Firecrawl keyless MCP	Initialized, listed three tools and successfully scraped example.com; no private data submitted
Harness MCP schema	Matched to upstream native plugin docs; no claim of a new full-profile installation on every OS
Hosted paid provider	Not freshly exercised in this build; needs the reader's chosen account/key
New RunPod deployment	Not launched: no new paid compute was provisioned for this release
RunPod idle-stop HTTP operation	Offline behavior tested; real stop requires an approved exact Pod and private key
Windows/Linux native setup	Documentation and prompts included; no fresh native-install claim

Use smoke-test-checklist.md for your installation. Do not treat a current documentation link as a successful live test. The original recording's model results and this release's fresh tests are different evidence.

Defensive use

Use this kit to learn, find weaknesses in systems you own or are explicitly authorized to test, and verify fixes. It does not grant permission to test any target.

Establish scope first

Record the system owner, how permission was obtained, the exact host or application, permitted actions, testing window, request limits, and stop conditions. Resolve missing or ambiguous permission before sending test requests. A public URL, valid login, or third-party API key is not permission to test a system.

Prefer a local lab or staging environment containing synthetic records. Start with non-destructive checks. Approval for one application does not include linked services, other accounts, neighboring hosts, or the provider's infrastructure.

Keep proof proportional

Collect only the evidence needed to confirm a finding. Do not download an entire database, display real credentials, alter another user's account, or retain unnecessary personal information. Redact findings before sharing them.

Stop when a check would cross the agreed boundary, affect availability, expose sensitive records, or require a destructive change. Record what happened and seek specific authorization for any proposed next step. Do not pursue persistence, stealth, or access beyond the test scope.

Protect your own environment

Keep credentials in a private local secret store or ignored configuration file. Do not paste them into prompts, commit them, or include them in screenshots. Use limited test credentials and revoke them after testing.

Run downloaded models with only the file, network, and tool access needed for the task. Instructions in a prompt do not enforce isolation. Verify the runtime's actual restrictions before connecting it to private files or production accounts.

Keep local endpoints bound to your own computer unless you deliberately configure authenticated remote access. Require authentication for a cloud endpoint. Review proposed hardware, hourly and storage charges, and stopping behavior before approving deployment.

Treat output as something to verify

AI can invent findings and incorrect commands. A scanner can miss vulnerabilities or report false positives. Review the evidence, reproduce the issue safely, apply a reviewed fix, and retest. No result in this kit is a certification, guarantee, or replacement for an independent security assessment.

Publication boundaries

Do not publish third-party credentials, private data, raw session logs, or working targets without permission. Check third-party licenses before redistributing code, images, or model files. Report accidental exposure using SECURITY.md.

Use and limitations

For education and authorized defensive use only. Test only systems you own or have explicit permission to test, within the agreed scope. Do not use these materials for unauthorized access, data theft, service disruption, or other unlawful activity.

These materials are provided “as is” and “as available,” without warranties to the extent permitted by applicable law. They are not legal advice, a professional security assessment, or a guarantee of security, accuracy, compatibility, availability, or fitness for a particular purpose. AI output can be incorrect or unsafe. Review commands, code, findings, and configuration changes before use.

You are responsible for permissions, backups, credentials, personal data, third-party terms, and usage charges. Third-party tools, models, availability, and pricing can change. Test in an isolated environment with synthetic data before considering production use.

To the extent permitted by applicable law, the creators and contributors disclaim liability for losses or damages arising from use of or reliance on these materials. Nothing in this notice excludes or limits liability, warranties, rights, or remedies that cannot lawfully be excluded or limited.